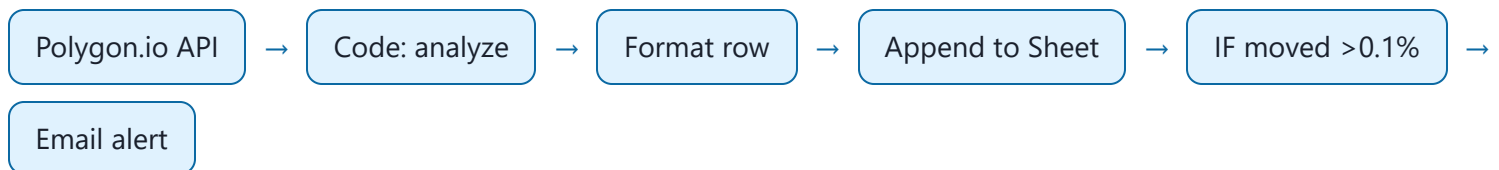


Activity 1 — n8n Stock Data Pipeline

This guide covers the **automation half** of Activity 1: an **n8n** workflow that fetches daily stock data from **Polygon.io**, analyzes it, logs it to a **Google Sheet**, and emails an alert. The **analysis half** (reading that sheet in Colab) is in the companion notebook.



Before you start

- A free **Polygon.io** account and API key.
- An **n8n** instance (cloud or self-hosted).
- A **Google Sheet** (and Gmail) connected to n8n.

API key safety: never share or commit your Polygon key. In the URL below, replace `YOUR_POLYGON_API_KEY` with your own key inside n8n only. If a key is ever exposed publicly, **rotate it** in your Polygon dashboard.

Data source — Polygon.io

Previous-day aggregate for a ticker:

```
https://api.polygon.io/v2/aggs/ticker/AAPL/prev?adjusted=true&apikey=YOUR_POLYGON_API_KEY
```

Tickers used in the demo: **AAPL** (Apple), **META** (Meta), **MSFT** (Microsoft), **GOOGL** (Google), **TSLA** (Tesla).

Step 1 — Code node: analyze the stock data

Extracts open/close/high/low/volume, computes daily return and volatility, and produces a trading signal.

N8N CODE NODE (JAVASCRIPT)

```
// Get the stock data from previous node
const stockData = $input.first().json;
const results = stockData.results[0]; // Polygon.io structure

// Extract key metrics (Polygon.io format)
const openPrice = results.o;
const closePrice = results.c;
const highPrice = results.h;
const lowPrice = results.l;
const volume = results.v;

// Calculate daily return percentage (open to close)
const dailyReturn = ((closePrice - openPrice) / openPrice) * 100;

// Calculate volatility (high-low range as % of close)
const volatility = ((highPrice - lowPrice) / closePrice) * 100;

// Generate simple trading signals
let signal = "HOLD";
let confidence = "LOW";

if (dailyReturn > 2) {
  signal = "STRONG BUY"; confidence = "HIGH";
} else if (dailyReturn > 0.5) {
  signal = "BUY"; confidence = "MEDIUM";
} else if (dailyReturn < -2) {
  signal = "STRONG SELL"; confidence = "HIGH";
} else if (dailyReturn < -0.5) {
  signal = "SELL"; confidence = "MEDIUM";
}

// Risk assessment
const riskLevel = volatility > 3 ? "HIGH" : volatility > 1.5 ? "MEDIUM" : "LOW";

return [{
  json: {
    symbol: "AAPL",
    date: new Date().toISOString().split('T')[0],
    timestamp: new Date().toISOString(),
    prices: { open: openPrice, close: closePrice, high: highPrice, low: lowPrice },
    volume: volume,
    analysis: {
      dailyReturn: Math.round(dailyReturn * 100) / 100,
      volatility: Math.round(volatility * 100) / 100,
```

```
    signal: signal, confidence: confidence, riskLevel: riskLevel
  },
  recommendation: `${signal} with ${confidence} confidence. Risk level: ${riskLevel}`
}
}];
```

Step 2 — Code node: shape the row for the Sheet

N8N CODE NODE (JAVASCRIPT)

```
const data = $input.first().json;

return [{
  json: {
    DATE: data.date,
    SYMBOL: data.symbol,
    CLOSE_PRICE: data.prices.close,
    DAILY_RETURN: data.analysis.dailyReturn + "%",
    SIGNAL: data.analysis.signal,
    RISK_LEVEL: data.analysis.riskLevel
  }
}];
```

Then add a **Google Sheets** → **Append row** node that writes these columns to your sheet.

Step 3 — IF node: only alert on real movement

```
{{ $json.analysis.dailyReturn }} is greater than 0.1
```

Step 4 — Email subject (HTML)

```
STOCK ALERT: {{ $('Append row in sheet').item.json.SYMBOL }} {{ $('Append row in sheet').item.json.DATE }}
```

Step 5 — Email body (HTML)

```
<h2>STOCK MOVEMENT ALERT</h2>

<p><strong>Symbol:</strong> {{ $('Append row in sheet').item.json.SYMBOL }}</p>
<p><strong>Current Price:</strong> ${{ $('Append row in sheet').item.json['CLOSE PRICE'] }}
</p>
<p><strong>Daily Return:</strong> {{ $('Append row in sheet').item.json['DAILY RETURN %'] }} %
</p>

<h3>SIGNAL: {{ $('Append row in sheet').item.json.SIGNAL }}</h3>
<p><strong>Confidence:</strong> {{ $('Code').item.json.analysis.confidence }}</p>
<p><strong>Risk Level:</strong> {{ $('Append row in sheet').item.json['RISK LEVEL'] }}</p>

<p><strong>Recommendation:</strong> {{ $('Code').item.json.recommendation }}</p>

<hr>
<p><em>Automated by n8n Data Pipeline</em><br>
<em>Generated at:</em> {{ $('Append row in sheet').item.json.DATE }}</p>
```

Next: let the pipeline run for a while so the sheet fills with rows, then open the **Activity 1 Colab notebook** and point it at your sheet to analyze the collected data.